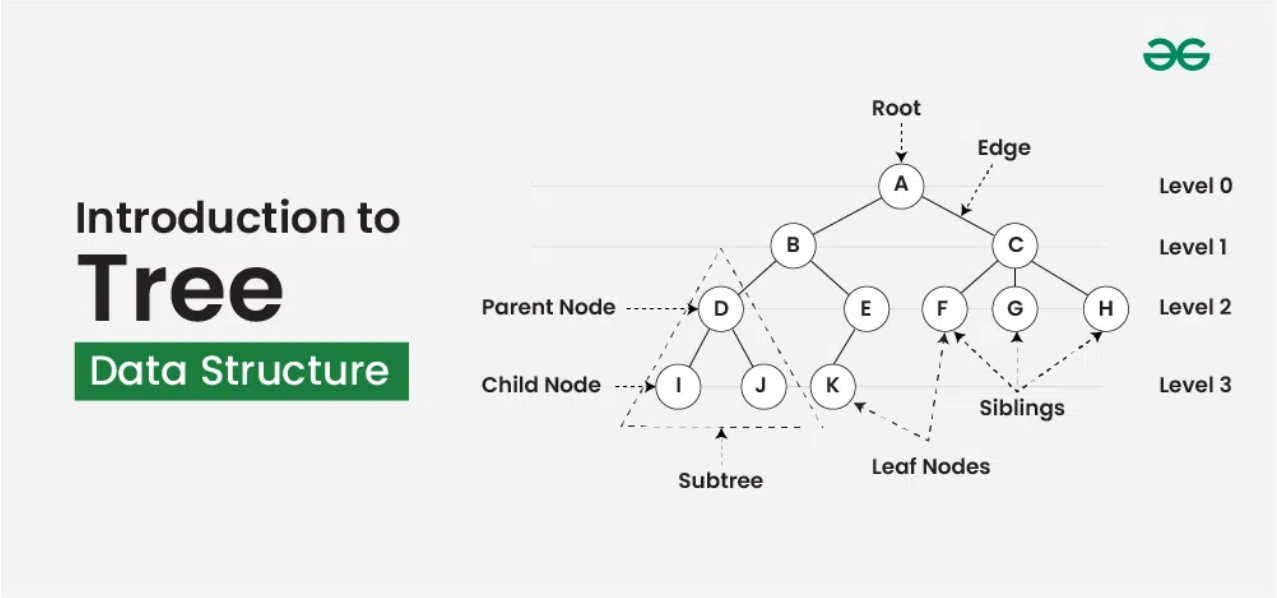
Introduction to Tree Data Structure

Last Updated : 04 Mar, 2025

Tree data structure is a hierarchical structure that is used to represent and organize data in the form of parent child relationship. The following are some real world situations which are naturally a tree.

- Folder structure in an operating system.
- Tag structure in an HTML (root tag the as html tag) or XML document.

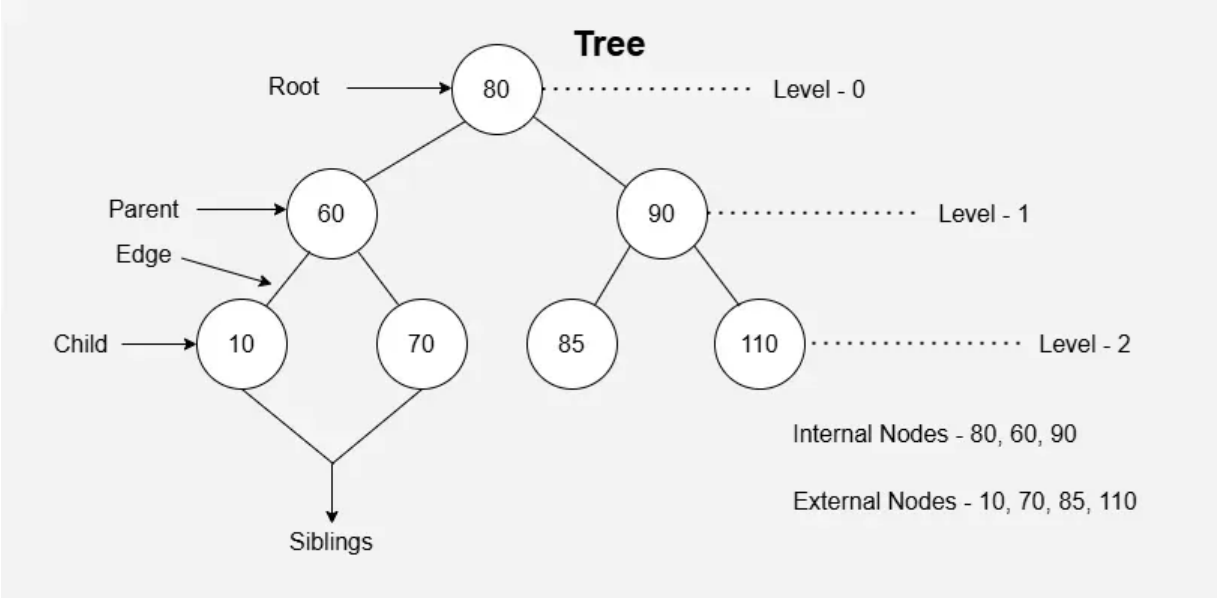
The topmost node of the tree is called the **root**, and the nodes below it are called the child nodes. Each node can have multiple child nodes, and these child nodes can also have their own child nodes, forming a recursive structure.



Basic Terminologies In Tree Data Structure:

- **Parent Node:** The node which is an immediate predecessor of a node is called the parent node of that node. **{B}** is the parent node of **{D, E}**.
- **Child Node:** The node which is the immediate successor of a node is called the child node of that node. Examples: **{D, E}** are the child nodes of **{B}.**
- **Root Node:** The topmost node of a tree or the node which does not have any parent node is called the root node. {A**}** is the root node of the tree. A non-empty tree must contain exactly one root node and exactly one path from the root to all other nodes of the tree.

- **Leaf Node or External Node:** The nodes which do not have any child nodes are called leaf nodes. **{I, J, K, F, G, H}** are the leaf nodes of the tree.
- **Ancestor of a Node:** Any predecessor nodes on the path of the root to that node are called Ancestors of that node. **{A,B}** are the ancestor nodes of the node **{E}**
- **Descendant:** A node x is a descendant of another node y if and only if y is an ancestor of x.
- **Sibling:** Children of the same parent node are called siblings. **{D,E}** are called siblings.
- **Level of a node:** The count of edges on the path from the root node to that node. The root node has level **0**.
- **Internal node:** A node with at least one child is called Internal Node.
- **Neighbour of a Node:** Parent or child nodes of that node are called neighbors of that node.
- **Subtree**: Any node of the tree along with its descendant.



Basic Terminologies In Tree

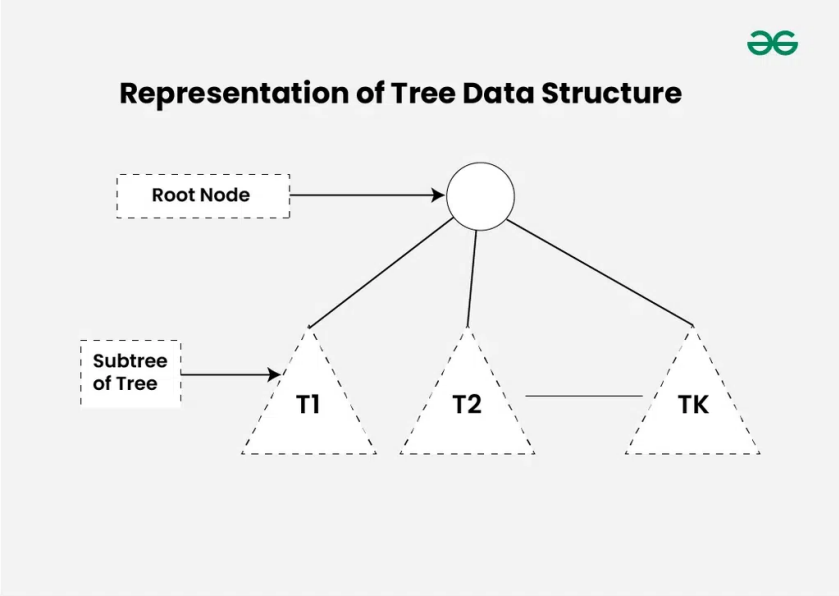
Why Tree is considered a non-linear data structure?

The data in a tree are not stored in a sequential manner i.e., they are not stored linearly. Instead, they are arranged on multiple levels or we can say it is a hierarchical structure. For this reason, the tree is considered to be a non-linear data structure.

We strongly recommend to study a Binary Tree first as a Binary Tree has structure and code implementation compared to a general tree.

Representation of Tree Data Structure:

A tree consists of a root node, and zero or more subtrees T_1, T_2, ... , T_k such that there is an edge from the root node of the tree to the root node of each subtree. Subtree of a node X consists of all the nodes which have node X as the ancestor node.



Representation of Tree Data Structure

Representation of a Node in Tree Data Structure:

A tree can be represented using a collection of nodes. Each of the nodes can be represented with the help of class or structs. Below is the representation of Node in different languages:

C++ C Java Python JavaScript

```
class Node {
public:
    int data;
    Node* first_child;
    Node* second_child;
    Node* third_child;
    .
    .
    .
```

```
    Node* nth_child;
};
```

Importance for Tree Data Structure:

One reason to use trees might be because you want to store information that naturally forms a hierarchy. For example, the file system on a computer: The DOM model of an HTML page is also tree where we have html tag as root, head and body its children and these tags, then have their own children.Please refer Applications, Advantages and Disadvantages of Tree for details.

Types of Tree data structures:

Tree data structure can be classified into three types based upon the number of children each node of the tree can have. The types are:

- **Binary tree:** In a binary tree, each node can have a maximum of two children linked to it. Some common types of binary trees include full binary trees, complete binary trees, balanced binary trees, and degenerate or pathological binary trees. Examples of Binary Tree are Binary Search Tree and Binary Heap.
- **Ternary Tree:** A Ternary Tree is a tree data structure in which each node has at most three child nodes, usually distinguished as “left”, “mid” and “right”.
- **N-ary Tree or Generic Tree:** Generic trees are a collection of nodes where each node is a data structure that consists of records and a list of references to its children(duplicate references are not allowed). Unlike the linked list, each node stores the address of multiple nodes.

Please refer Types of Trees in Data Structures for details.

Basic Operations Of Tree Data Structure:

- **Create** – create a tree in the data structure.
- **Insert** – Inserts data in a tree.
- **Search** – Searches specific data in a tree to check whether it is present or not.
- **Traversal**:

- Depth-First-Search Traversal
- Breadth-First-Search Traversal

Implementation of Tree Data Structure:

C++ Java Python C# JavaScript

```cpp
// C++ program to demonstrate some of the above
// terminologies
#include <bits/stdc++.h>
using namespace std;

// Function to add an edge between vertices x and y
void addEdge(int x, int y, vector<vector<int> >& adj)
{
    adj[x].push_back(y);
    adj[y].push_back(x);
}

// Function to print the parent of each node
void printParents(int node, vector<vector<int> >& adj,
                  int parent)
{
    // current node is Root, thus, has no parent
    if (parent == 0)
        cout << node << "->Root" << endl;
    else
        cout << node << "->" << parent << endl;

    // Using DFS
    for (auto cur : adj[node])
        if (cur != parent)
            printParents(cur, adj, node);
}

// Function to print the children of each node
void printChildren(int Root, vector<vector<int> >& adj)
{
    // Queue for the BFS
    queue<int> q;
```

```
    // pushing the root
    q.push(Root);

    // visit array to keep track of nodes that have been
    // visited
    int vis[adj.size()] = { 0 };

    // BFS
    while (!q.empty()) {
        int node = q.front();
        q.pop();
        vis[node] = 1;
        cout << node << "-> ";
        for (auto cur : adj[node])
            if (vis[cur] == 0) {
                cout << cur << " ";
                q.push(cur);
            }
        cout << endl;
    }
}

// Function to print the leaf nodes
void printLeafNodes(int Root, vector<vector<int> >& adj)
{
    // Leaf nodes have only one edge and are not the root
    for (int i = 1; i < adj.size(); i++)
        if (adj[i].size() == 1 && i != Root)
            cout << i << " ";
    cout << endl;
}

// Function to print the degrees of each node
void printDegrees(int Root, vector<vector<int> >& adj)
{
    for (int i = 1; i < adj.size(); i++) {
        cout << i << ": ";

        // Root has no parent, thus, its degree is equal to
        // the edges it is connected to
        if (i == Root)
            cout << adj[i].size() << endl;
```

```
        else
            cout << adj[i].size() - 1 << endl;
    }
}
// Driver code
int main()
{
    // Number of nodes
    int N = 7, Root = 1;
    // Adjacency list to store the tree
    vector<vector<int> > adj(N + 1, vector<int>());
    // Creating the tree
    addEdge(1, 2, adj);
    addEdge(1, 3, adj);
    addEdge(1, 4, adj);
    addEdge(2, 5, adj);
    addEdge(2, 6, adj);
    addEdge(4, 7, adj);

    // Printing the parents of each node
    cout << "The parents of each node are:" << endl;
    printParents(Root, adj, 0);

    // Printing the children of each node
    cout << "The children of each node are:" << endl;
    printChildren(Root, adj);

    // Printing the leaf nodes in the tree
    cout << "The leaf nodes of the tree are:" << endl;
    printLeafNodes(Root, adj);

    // Printing the degrees of each node
    cout << "The degrees of each node are:" << endl;
    printDegrees(Root, adj);

    return 0;
}
```

Output

```
The parents of each node are:
1->Root
2->1
5->2
6->2
3->1
4->1
7->4
The children of each node are:
1-> 2 3 4
2-> 5 6
3->
4-> 7
5->
6->
7->
The leaf nodes of the tree are:
3 5 6 7
The degrees o...
```

Properties of Tree Data Structure:

Number of edges: An edge can be defined as the connection between two nodes. If a tree has N nodes then it will have (N-1) edges. There is only one path from each node to any other node of the tree.

Depth of a node: The depth of a node is defined as the length of the path from the root to that node. Each edge adds 1 unit of length to the path. So, it can also be defined as the number of edges in the path from the root of the tree to the node.

- **Height of a node:** The height of a node can be defined as the length of the longest path from the node to a leaf node of the tree.
- **Height of the Tree:** The height of a tree is the length of the longest path from the root of the tree to a leaf node of the tree.
- **Degree of a Node:** The total count of subtrees attached to that node is called the degree of the node. The degree of a leaf node must be **0**. The degree of a tree is the maximum degree of a node among all the nodes in the tree.

Easy Problem On Tree in JavaScript

- Inorder Traversal of Binary Tree
- Preorder Traversal of Binary Tree
- Postorder Traversal of Binary Tree
- Level Order Tree Traversal
- Height or Depth of a Binary Tree

Medium Problem On Tree in JavaScript

- Connect nodes at same level
- Nodes at given distance in a Binary Tree
- Binary Tree to Doubly Linked List
- Maximum sum path between two leaf
- K-Sum Paths
- Boundary Traversal

Tree Data Structure

Next Article

Tree Traversal Techniques

Similar Reads

Introduction to Tree Data Structure

Tree data structure is a hierarchical structure that is used to represent and organize data in the form of parent child relationship. The following are some real world situations which are naturally a tree.Folder...

15+ min read

Tree Traversal Techniques

Tree Traversal techniques include various ways to visit all the nodes of the tree. Unlike linear data structures (Array, Linked List, Queues, Stacks, etc) which have only one logical way to traverse them,...

7 min read

Applications of tree data structure

A tree is a type of data structure that represents a hierarchical relationship between data elements, called nodes. The top node in the tree is called the root, and the elements below the root are called child node...

4 min read

Advantages and Disadvantages of Tree

Tree is a non-linear data structure. It consists of nodes and edges. A tree represents data in a hierarchical organization. It is a special type of connected graph without any cycle or circuit.Advantages of...

2 min read

Difference between an array and a tree

Array:An array is a collection of homogeneous(same type) data items stored in contiguous memory locations. For example, if an array is of type â€œintâ€ , it can only store integer elements and cannot...

3 min read

Inorder Tree Traversal without Recursion

Given a binary tree, the task is to perform in-order traversal of the tree without using recursion.Example:Input:Output: 4 2 5 1 3Explanation: Inorder traversal (Left->Root->Right) of the tree i...

8 min read

Types of Trees in Data Structures

A tree in data structures is a hierarchical data structure that consists of nodes connected by edges. It is used to represent relationships between elements, where each node holds data and is connected to oth...

4 min read

Generic Trees (N-ary Tree)

Binary Tree

Ternary Tree